

KYAMBOGO UNIVERSITY

FACULTY OF ENGINEERING · DEPARTMENT OF ELECTRICAL AND ELECTRONICS

TETE 3102: MICROCONTROLLERS AND APPLICATIONS

LABORATORY REPORT: TASK 6

Systematic I2C Bus Fault Finding & Diagnostic Matrix

Mukama Robert (26/U/ETE/21285/PE)
Nabudere Zerubabel (25/U/ETE/05465/PE)
Nanfuka Shakirah (25/U/ETE/05468/PE)
Kidega Wilkie Cephas (25/U/ETE/05460/PE)

Group Number: Group 4
Platforms: STM32F411RE & Arduino Uno
Course Supervisor: Dr. Mugerwa Dickson PhD

OBJECTIVE & LAB FOCUS

The objective of this laboratory task is to develop and apply a systematic troubleshooting methodology for identifying, diagnosing, and resolving physical, electrical, and firmware faults on an I^2C bus. Using return codes, bus scanners, and mixed-signal oscilloscope probing, students isolate six real-world failure modes and document corrective actions in an engineering fault matrix.

1 Troubleshooting Methodology

Diagnosing I^2C anomalies follows a three-stage hierarchical workflow:

1. **Layer 1: Software Return Status & Error Flags:** Interrogate return codes from `HAL_I2C_GetError()` or Arduino `Wire.endTransmission()` to distinguish between address NACK, data NACK, and timeouts.
2. **Layer 2: Bus Scanner Interrogation:** Determine whether individual devices respond, the entire bus is dead, or ghost devices appear across reserved address ranges.
3. **Layer 3: Physical Signal Integrity Inspection:** Probe SDA and SCL using an oscilloscope to verify idle voltage rails, rise times, ground reference integrity, and line transitions.

2 Comprehensive Fault Diagnostic Matrix

3 In-Depth Fault Analysis: The Bus Recovery Routine

HARDWARE & PROTOCOL TROUBLESHOOTING NOTE

The "Stuck SDA" Lockup Condition: A notorious I^2C failure occurs when the master resets mid-byte while a slave is transmitting a logic 0. Because the slave is waiting for clock pulses that never arrive, it holds SDA LOW indefinitely. The master powers up, senses SDA LOW, detects a "Bus Busy" condition, and refuses to initiate communication.

Software Bus Clearing Algorithm (Clock Toggling Recovery)

When `HAL_I2C_Init()` returns `HAL_BUSY`, execute this 9-pulse clock clearing sequence:

Table 1: Systematic Diagnostic Matrix for the Six Injected Bus Faults

Injected Fault	Observed Symptoms	Return Code / Error Flag	Test Equipment Observation	Corrective Action (Fix)
1. Wrong Address (`0x3F` instead of `0x27`)	LCD remains completely uninitialized and blank; scanner finds device only at `0x27`.	STM32: `HAL_ERROR` (`HAL_I2C_ERROR_AF`). Arduino: Code **2** (`NACK on addr`).	Scope reveals 8 address bits pulsed, followed by SDA remaining HIGH on 9th clock (no slave pulls down).	Run bus scanner to determine true physical address; update `#define LCD_ADDR` to matching address.
2. Address Not Shifted on STM32 (`0x27` passed instead of `0x27 « 1`)	STM32 program fails immediately on first transmit; Arduino code works perfectly with identical hardware.	STM32: `HAL_ERROR` (`HAL_I2C_ERROR_AF`).	Scope decodes the wire address as `0x13` with $R/\overline{W} = 1$ (Read). Slave ignores transmission.	Shift the 7-bit address left by 1 bit: '(0x27 « 1) = 0x4E' before passing to STM32 HAL functions.
3. SDA and SCL Lines Swapped	Bus scanner reports no devices found or hangs; LCD and RTC completely unresponsive.	STM32: `HAL_BUSY` or `HAL_TIMEOUT`. Arduino: Code **4** or **5**.	Scope shows clock pulses on the slave's SDA pin while SCL remains static high.	Trace physical breadboard wiring; swap jumper connections to ensure MCU SCL → Slave SCL and SDA → SDA.
4. One / Both Pull-Up Resistors Removed	With one removed: high-speed data corrupts. With both removed: bus completely dead.	STM32: `HAL_TIMEOUT` or `HAL_I2C_ERROR_TIMEOUT`.	Scope shows lines floating down to 0 V or displaying severely deformed rise times ($> 5\,\mu$s).	Install external 2.2 kΩ to 4.7 kΩ pull-up resistors from SDA and SCL to the 3.3 V or 5 V rail.
5. Address Conflict (Two PCF8574 backpacks set to `0x27`)	Both displays show garbled, flickering characters; cursor commands fail unpredictably.	May return `HAL_OK` (both ACK), but data corruption occurs.	Both devices pull SDA low simultaneously during data read; bus contentions observable on scope.	Solder address jumper A0 on the second backpack to change its hardware address to `0x26` or `0x25`.
6. Voltage Mismatch (5V pull-ups to non-tolerant pin)	Communication may function briefly; risk of excessive leakage current or pin clamp conduction.	Intermittent bit errors due to disparate logic thresholds.	Multimeter measures 5.0 V on STM32 pin. Scope shows logic high levels overshooting 3.6 V.	Insert a bidirectional MOSFET logic-level converter (3.3 V ↔ 5.0 V) or pull resistors to 3.3 V.

Listing 1: Robust I2C Bus Recovery Routine for STM32

```c
void I2C_Bus_Recovery(void) {
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* 1. De-initialize I2C1 peripheral */
    HAL_I2C_DeInit(&hi2c1);

    /* 2. Configure SCL (PB8) and SDA (PB9) as GPIO Open-Drain Outputs */
    __HAL_RCC_GPIOB_CLK_ENABLE();
    GPIO_InitStruct.Pin = GPIO_PIN_8 | GPIO_PIN_9;
    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_OD;
    GPIO_InitStruct.Pull = GPIO_PULLUP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_HIGH;
    HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

    /* 3. If SDA is held low by a locked slave, clock SCL up to 9 times */
    for (uint8_t i = 0; i < 9; i++) {
        if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_9) == GPIO_PIN_SET) {
            break; // Slave released SDA!
        }
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_RESET);
        for(volatile int d=0; d<100; d++);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_SET);
        for(volatile int d=0; d<100; d++);
    }

    /* 4. Generate a clean STOP condition manually */
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, GPIO_PIN_RESET);
```

```
    for(volatile int d=0; d<100; d++);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_SET);
    for(volatile int d=0; d<100; d++);
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_9, GPIO_PIN_SET); // SDA rises while SCL high

    /* 5. Re-initialize I2C1 peripheral in CubeMX mode */
    MX_I2C1_Init();
}
```

4 Conclusion

Structured fault isolation confirms that software return codes provide the first line of defense, but physical test instruments (oscilloscopes and multimeters) are indispensable for diagnosing analog pull-up, wiring, and bus lockup anomalies.